

Mandate - Final Submission Deck

Encode x Arc Programmable Money Hackathon - DeFi track - Final submission (2026-08-10)

Slide 1 - Mandate

Treasury rules in plain English, executed as deterministic onchain flows on Arc.

"route 10% of every incoming payment to savings" - typed once, enforced forever.

An AI interpreter reads the rule once, then leaves the loop permanently. **AI touches money zero times.**

Live app: <https://mandate-olive.vercel.app>

Repo: github.com/danielmartin227/mandate

Slide 2 - The problem

- Onchain treasuries are managed by hand: someone watches balances, splits revenue, tops up chains.
- Automation today means either writing custom contracts (slow, risky) or trusting an AI agent with execution (unauditable: the same prompt can produce different transfers tomorrow).
- Finance teams already work with mandates: fixed instructions, executed exactly. Crypto treasuries deserve the same guarantee.

Slide 3 - How it works: sentence to template, then AI leaves

1. **Compile (once, with AI):** one constrained LLM call maps the sentence to one of exactly three fixed templates - Split, Sweep, Bridge - and fills the parameters. Ambiguity is refused, never guessed. A human confirms the plain-English readback before anything is deployed.
2. **Deploy:** parameters land in a thin, Blockscout-verified Solidity template on Arc. `execute()` is permissionless.
3. **Run (forever, no AI):** a keeper watches incoming USDC through Arc's EIP-7708 system emitter and ticks scheduled rules, then calls `execute()`. Same event in, same transfer out, every time.

The rule survives even if our backend dies: anyone can call `execute()`.

Slide 4 - Differentiation

	arc-fintech (Circle sample)	ArcFlux (prior hackathon)	Mandate
Standing rules	none, manual buttons	scheduled push payments	reactive treasury policy on incoming funds
AI's role	none	GPT-4 parses every command; Guardian Agent scores every transaction	compiled out after setup, zero AI at execution
Onchain contracts	none	none (custodial wallets + Python backend)	verified templates, permissionless execute()
Cross-chain	bridging UI	explicitly none	CCTP V2 called by the contract itself, Arc to Base

Slide 5 - Tech stack

- **Arc Testnet** (chain 5042002): USDC-native gas, sub-second finality
- **USDC**: the treasury asset; Arc's dual native (18d) and ERC-20 (6d) interfaces both handled
- **CCTP V2**: Arc (domain 26) to Base Sepolia (domain 6), attestation via Circle's Iris API, `depositForBurn` called by the rule contract
- **EIP-7708 watcher**: Arc's system emitter logs every USDC movement, so one filter catches native and ERC-20 incoming payments (an Arc-specific capability)
- **solc-js + viem**: template compilation, deployment, Blockscout verification
- **Next.js**: compile, preview, confirm, and the live execution feed
- TypeScript throughout

Slide 6 - Shipped and verifiable

All three templates are live and source-verified on Arc Testnet:

Template	Address
Split	0x005861b9bc42957178aa2e2e47adf63eb5dcb915
Sweep	0xa10269b7df3fa969381f9bfd251634a8a3abd751
Bridge	0x11016575c46b62a656c6e5dd261430ad97f9aec3

- **Sentence to live rule:** the Split rule above was compiled by the interpreter from a typed sentence and confirmed by a human before deploy.
- **Real payment, real enforcement:** 0.7 USDC arrived, 0.035 USDC routed to savings, tx `0x15203d9a...97e708a8`.
- **Full CCTP round trip:** 0.5 USDC burned on Arc (`0x6706f936...a69af779`) and minted on Base Sepolia (`0x2e1a4bd7...738e93ed`). `destinationCaller` is zero, so anyone can submit the mint.

Not used: Circle App Kits. Nothing in the money path is simulated. All funds are Arc testnet USDC.

Slide 7 - Why Arc makes this possible

- On every other EVM chain, native-token payments leave **no logs**, so a splitter must poll balances or trace blocks.
- Arc emits an ERC-20-style `Transfer` log from the system emitter `0xffff...fffe` for **every** USDC movement, native or ERC-20 (EIP-7708).
- One log filter gives a complete, deterministic feed of incoming treasury payments.
- That single property is what lets a fixed onchain rule react to real revenue with no oracle, no indexer, and no AI.